2013 IEEE 25th International Conference on Tools with Artificial Intelligence

Modulo Based CNF Encoding of Cardinality Constraints and Its Application to MaxSAT Solvers

Toru Ogawa† YangYang Liu†
†*Graduate School of Information Science and Electrical Engineering*
Kyushu University
Fukuoka, Japan 819-0395
{*ogawa,liu*}*@ar.inf.kyushu-u.ac.jp*

Ryuzo Hasegawa‡ Miyuki Koshimura‡ Hiroshi Fujita‡
‡*Faculty of Information Science and Electrical Engineering*
Kyushu University
Fukuoka, Japan 819-0395
{*hasegawa,koshi,fujita*}*@inf.kyushu-u.ac.jp*

***Abstract*— *Totalizer* (TO) by Bailleux et al. and *Half Sorting Network* (HS) by Asin et al. are typical CNF encoding methods of cardinality constraint. The former is based on unary adder, while the latter is based on odd-even merge. Although TO is inferior to HS in terms of the number of clauses, TO is superior to HS in terms of the number of variables. We propose a new method called *Modulo Totalizer* (MTO) to overcome the disadvantage of TO. As an application, we have developed a partial MaxSAT solver with MTO. Preliminary experimental results show that our MTO based MaxSAT solver is comparable to or surpass the conventional TO based maxsat solvers.**

***Keywords*- CNF Encoding; Modulo Calculation; MaxSAT; Cardinality Constraints; K-Cardinality**

I. Introduction

Over the last decade we have witnessed a dramatic speed-up of Boolean Satisfiability (SAT) solvers[1], [2], [9]: problems with thousands of variables are now solved in milliseconds by state-of-the-art SAT solvers. Many constraints satisfaction problems have been solved by transforming (encoding) them into SAT instances which are in conjunctive normal form (CNF). Therefore quite some work has been done to finding good propositional encodings for many kinds of constraints. A particularly important constraint is the cardinality constraints, i.e. constraints of the form $x_1 + \ldots + x_n \prec k$ where k is a natural number and the relation $\prec$ is one of $\{<, \leq, >, \geq, =\}$.

There are many works[3], [4], [10], [11], [12] that describe techniques to encode cardinality constraints to CNF. The starting points for this work are Totalizer by Bailleux and Boufkhad[3] and Half Sorting Network by Asín et al[4]. In both encodings, the sum is represented by the unary notation. Totalizer requires $O(n^2)$ clauses with $O(n \log n)$ variables, while Half Sorting Network requires $O(n \log^2 n)$ clauses with $O(n \log^2 n)$ variables to encode $x_1 + \ldots + x_n \prec k$. Half Sorting Network is superior to Totalizer in terms of the number of clauses, but inferior to Totalizer in terms of the number of variables. Moreover, the critical path of Half Sorting Network is longer than that of Totalizer. This may cause a delay of constraints propagation.

We developed a partial MaxSAT solver QMaxSAT[6] which makes use of Totalizer. QMaxSAT won both crafted and industrial subcategories of Partial Max-SAT category in Max-SAT 2012[8]. This paper proposes a Modulo Totalizer that decreases the number of clauses in Totalizer. The Modulo Totalizer is based on modular arithmetic where an integer n $(= p \times q + r)$ is denoted as $q|_p r$ using modulus p. The Modulo Totalizer requires $O(n^{3/2})$ clauses with $O(n \log n)$ variables when $p = n^{1/2}$. We evaluate the proposed encoding by solving MaxSAT instances with QMaxSAT using Modulo Totalizer.

II. CNF encoding of cardinality constraints

MaxSAT solvers that utilize ordinal SAT solvers as inference engines are characterized by searching for MaxSAT solutions by repeatedly applying the SAT solver to the given problems. When the problems are unsatisfiable, ordinal SAT solvers will only answer "it is unsatisfiable". To get more useful information even in this case, we add to the original clause Cl_i a *blocking variable* b_i. The number k of blocking variables that are assigned 1 represents the number of unsatisfied clauses. In MaxSAT, the cardinality constraint for this is represented by $\Sigma_{i=1}^{n} b_i < k$, which gives constraints on the number of blocking variables that are assigned 1 in the following reasoning. In this section, we introduce CNF encoding schemes of cardinality constraints: Totalizer by Bailleux et al. and Half Sorting Network by Asin et al. as well as K-Cardinality versions of both.

A. Totalizer

Totalizer (TO) [3] proposed by Bailleux et al. is a CNF encoding of cardinality constraints based on a unary representation of integers. Here the value of an integer x such that $0 \leq x \leq n$ is represented by a sequence of 1 x times followed by 0 $n - x$ times. A TO generates n propositional variables according to n blocking variables. These variables have the following properties.

1082-3409/13 $31.00 © 2013 IEEE
DOI 10.1109/ICTAI.2013.13

IEEE computer society

Authorized licensed use limited to: b-on: Universidade de Lisboa Reitoria. Downloaded on October 02,2025 at 15:05:38 UTC from IEEE Xplore. Restrictions apply.

1) When p blocking variables are fixed to 1, $o_i(i = 1 \dots p)$ becomes 1.
2) When q blocking variables are fixed to 0, $o_i(i = n - q + 1 \dots n)$ becomes 0.

By the above properties, the cardinality constraint $\Sigma_{i=1}^{n} o_i < k$ is satisfied when o_k becomes 0.

The basic procedure for TO is a *Unary Adder* (UA) shown in Figure 1. Given two sequences $\langle a_1 \dots a_m \rangle$ and $\langle b_1 \dots b_n \rangle$ of variables, the UA produces a sequence $\langle r_1 \dots r_{m+n} \rangle$ of variables and a CNF formula Φ representing the addition $\alpha + \beta = \sigma$ in unary, where $0 \leq \alpha \leq m$, $0 \leq \beta \leq n$, and $0 \leq \sigma \leq m + n$. Note that in Figure 1, only the clauses to send 1 to the left, representing $\sigma \geq \alpha + \beta$, are generated.

As shown in Figure 2, the TO divides an input sequence $\langle i_1 \dots i_n \rangle$ into $\langle i_1 \dots i_{\lfloor n/2 \rfloor} \rangle$ and $\langle i_{\lfloor n/2 \rfloor + 1} \dots i_n \rangle$, and recursively calls two TOs that take as input either of the subsequences. Then with a UA, the TO produces a sequence $\langle o_1 \dots o_n \rangle$ of variables and a CNF formula Φ representing the addition of the results of two TO calls.

$$UA(\langle a_1 \dots a_m \rangle, \langle b_1 \dots b_n \rangle) = (\langle r_1 \dots r_{m+n} \rangle, \Phi)$$
$$\textbf{where} \quad \Phi = \bigwedge_{i=0}^{m} \bigwedge_{j=0}^{n} (\overline{a_i} \vee \overline{b_j} \vee r_{i+j}) \quad a_0 = b_0 = r_0 = 1$$

Figure 1. Unary Adder.

$$TO(\langle i_1 \dots i_n \rangle) = (\langle o_1 \dots o_n \rangle, \Phi_1 \wedge \Phi_2 \wedge \Phi) \quad \textbf{where}$$
$$TO(\langle i_1 \dots i_{\lfloor n/2 \rfloor} \rangle) = (\langle a_1 \dots a_{\lfloor n/2 \rfloor} \rangle, \Phi_1),$$
$$TO(\langle i_{\lfloor n/2 \rfloor + 1} \dots i_n \rangle = \langle b_1 \dots b_{n - \lfloor n/2 \rfloor} \rangle, \Phi_2),$$
$$UA(\langle a_1 \dots a_{\lfloor n/2 \rfloor} \rangle, \langle b_1 \dots b_{n - \lfloor n/2 \rfloor} \rangle) = (\langle o_1 \dots o_n \rangle, \Phi)$$
$$TO(\langle a \rangle) = \langle a \rangle \ (n = 1)$$

Figure 2. Totalizer.

B. Half Merging and Half Sorting Networks

Half Merging Network (HM) and *Half Sorting Network* (HS) [4] are a CNF encoding of cardinality constraints proposed by Asin et al. The Figure 3 shows the procedure for HM. The HM divides each of sorted sequences $\langle a_1 \dots a_n \rangle$ and $\langle b_1 \dots b_n \rangle$ into odd and even parts, and recursively calls two HMs. Note that in Figure 3, only the clauses to send 1 to the left are generated like UA. The Figure 4 shows the procedure for HS. The HS divides an input sequence $\langle i_1 \dots i_{2n} \rangle$ into $\langle i_1 \dots i_n \rangle$ and $\langle i_{n+1} \dots i_{2n} \rangle$, and recursively calls two HSs. Then with a HM, the HS generates a CNF formula Φ representing the merge of sorted sequences output from two HS calls.

$$HM(\langle a_1 \dots a_n \rangle, \langle b_1 \dots b_n \rangle) =$$
$$(\langle d_1 c_2 \dots c_{2n-1} e_n \rangle, \Phi_1 \wedge \Phi_2 \wedge \Phi) \quad \textbf{where}$$
$$HM(\langle a_1 a_3 \dots a_{n-1} \rangle, \langle b_1 b_3 \dots b_{n-1} \rangle) = (\langle d_1 \dots d_n \rangle, \Phi_1),$$
$$HM(\langle a_2 a_4 \dots a_n \rangle, \langle b_2 b_4 \dots b_n \rangle) = (\langle e_1 \dots e_n \rangle, \Phi_2),$$
$$\Phi = \bigwedge_{i=1}^{n-1} (\overline{d_{i+1}} \vee \overline{e_i} \vee c_{2i+1}) \wedge (\overline{d_{i+1}} \vee c_{2i}) \wedge (\overline{e_i} \vee c_{2i})$$
$$HM(\langle a \rangle, \langle b \rangle) = (\langle c_1 c_2 \rangle, (\overline{a} \vee \overline{b} \vee c_2) \wedge (\overline{a} \vee c_1) \wedge (\overline{b} \vee c_1))$$

Figure 3. Half Merging NetWork.

$$HS(\langle a_1 \dots a_{2n} \rangle) = (\langle o_1 \dots o_{2n} \rangle, \Phi_1 \wedge \Phi_2 \wedge \Phi) \quad \textbf{where}$$
$$HS(\langle a_1 \dots a_n \rangle) = (\langle d_1 \dots d_n \rangle, \Phi_1),$$
$$HS(\langle a_{n+1} \dots a_{2n} \rangle) = (\langle e_1 \dots e_n \rangle, \Phi_2),$$
$$HM(\langle d_1 \dots d_n \rangle, \langle e_1 \dots e_n \rangle) = (\langle o_1 \dots o_{2n} \rangle, \Phi)$$
$$HS(\langle a, b \rangle) = HM(\langle a \rangle, \langle b \rangle)$$

Figure 4. Half Sorting Network.

C. K-Cardinality

In cardinality constraints $\Sigma_{i=1}^{n} b_i < k$, it is often the case that n is much larger than k. If we are interested in the (maximal) k bits of the n output variables, the size of CNF encoding of cardinality constraints can be reduced by omitting the parts related to output variables $o_i(k < i \leq n)$. This encoding technique is called *K-Cardinality Networks*[4]. We introduce schemes that apply the above improvements to TO and HS below.

The procedures for *K-Cardinality Totalizer* (KTO) are shown in Figures 5 and 6. The difference with an ordinal TO is in that the KTO uses a *Simplified Unary Adder* (SUA) that only performs the unary addition for $i + j \leq k$.

$$SUA(\langle a_1 \dots a_m \rangle, \langle b_1 \dots b_n \rangle) = (\langle r_1 \dots r_{min(m+n,k)} \rangle, \Phi)$$
$$\textbf{where} \quad \Phi = \bigwedge_{i=0}^{min(k,m)} \bigwedge_{j=0}^{min(k-i,n)} (\overline{a_i} \vee \overline{b_j} \vee r_{i+j}) \quad a_0 = b_0 = r_0 = 1$$

Figure 5. Simplified Unary Adder.

$$KTO(\langle i_1 \dots i_n \rangle, k) = (\langle o_1 \dots o_n \rangle, \Phi_1 \wedge \Phi_2 \wedge \Phi) \ (n > k)$$
$$\textbf{where} \quad KTO(\langle i_1 \dots i_{\lfloor n/2 \rfloor} \rangle, k) = (\langle a_1 \dots a_m \rangle, \Phi_1),$$
$$KTO(\langle i_{\lfloor n/2 \rfloor + 1} \dots i_n \rangle, k) = (\langle b_1 \dots b_l \rangle, \Phi_2), \ m \leq k$$
$$SUA(\langle a_1 \dots a_m \rangle, \langle b_1 \dots b_l \rangle) = (\langle o_1 \dots o_k \rangle, \Phi) \ l \leq k$$
$$SUA(\langle a_1 \dots a_n \rangle, k) = TO(\langle a_1 \dots a_n \rangle) \ (n \leq k)$$

Figure 6. K-Cardinality Totalizer.

Authorized licensed use limited to: b-on: Universidade de Lisboa Reitoria. Downloaded on October 02,2025 at 15:05:38 UTC from IEEE Xplore. Restrictions apply.

In a similar way, the procedure for *K-Cardinality Half Sorting Network* (KHS) is defined by using a *Simplified Merging Network* (SM) that suppresses the output to k, as shown in Figures 7 and 8.

$SM(\langle a_1 \dots a_n \rangle, \langle b_1 \dots b_n \rangle)$
$= (\langle d_1 c_2 \dots c_{n+1} \rangle, \Phi_1 \wedge \Phi_2 \wedge \Phi)$ **where**
$SM(\langle a_1 a_3 \dots a_{n-1} \rangle, \langle b_1 b_3 \dots b_{n-1} \rangle) = (\langle d_1 \dots d_{\frac{n}{2}+1} \rangle, \Phi_1)$,
$SM(\langle a_2 a_4 \dots a_n \rangle, \langle b_2 b_4 \dots b_n \rangle) = (\langle e_1 \dots e_{\frac{n}{2}+1} \rangle, \Phi_2)$,
$\Phi = \bigwedge_{i=1}^{n/2} (\overline{d_{i+1}} \vee \overline{e_i} \vee c_{2i+1}) \wedge (\overline{d_{i+1}} \vee c_{2i}) \wedge (\overline{e_i} \vee c_{2i})$
$SM(\langle a \rangle, \langle b \rangle) = HM(\langle a \rangle, \langle b \rangle)$

Figure 7. Simplified Merging NetWork.

$KHS(\langle a_1 \dots a_n \rangle, k)$ $\quad n = m \times k, k = 2^r$
$= (\langle o_1 \dots o_k \rangle, \Phi_1 \wedge \Phi_2 \wedge \Phi)$ **where**
$KHS(\langle a_1 \dots a_{\lfloor m/2 \rfloor \times k} \rangle) = (\langle d_1 \dots d_k \rangle, \Phi_1)$,
$KHS(\langle a_{(\lfloor m/2 \rfloor \times k)+1} \dots a_n \rangle) = (\langle e_1 \dots e_k \rangle, \Phi_2)$,
$SM(\langle d_1 \dots d_k \rangle, \langle e_1 \dots e_k \rangle) = (\langle o_1 \dots o_{k+1} \rangle, \Phi)$
$KHS(\langle a_1 \dots a_k \rangle, k) = HS(\langle a_1 \dots a_k \rangle)$

Figure 8. K-Cardinality Half Sorting Network.

III. Exact counting the number of clauses and variables

In this section, we present a method to count the exact number of clauses and variables generated by TO, HS and K-Cardinality versions for both. A binary recursive procedure that takes an input sequence of n variables, such as TO and HS, invokes two recursive procedures with input sequences of $n/2$ variables. Then it generates a CNF formula by making some specific operation on their results. In general, the number of CNF components (clauses, variables) generated by the binary recursive procedure for $n = 2^r$ is given by $F(r) = 2F(r-1) + g(r)$. Here, $g(r)$ denotes the number of CNF components generated by making some operation on the results of the recursive procedures for $n/2$. The above recurrence is expanded as follows.

$$F(r) = 2^r C + \sum_{i=1}^{r} 2^{r-i} g(i) \ (r \geq 1), \quad F(0) = C \qquad (1)$$

In the sequel, we denote by $\#C(P)$ the number of clauses generated by procedure P, and by $\#V(P)$ the number of variables generated by procedure P. Here, we assume that both TO and HS generate only the clauses to send 1 to the left.

A. CNF size of Totalizer

At first, we count the size of a CNF formula generated by TO. Given inputs $\langle a_1 \dots a_{m_1} \rangle$ and $\langle b_1 \dots b_{m_2} \rangle$, the UA generates clauses for every couple $(i, j)(0 \leq i \leq m_1, 0 \leq j \leq m_2)$ except for $i = j = 0$, and as many output variables as input variables. Thus we have:

$\#C(UA) = m_1 m_2 + m_1 + m_2$
$\#V(UA) = m_1 + m_2$

We denote by TO_n a TO that has an input $\langle i_1 \dots i_n \rangle$. For $n = 2^r$, TO_n applies a UA to two $TO_{n/2}$s so that $g(r) = (2^{r-1})^2 + 2^r$ with respect to clauses, and $g(r) = 2^r$ with respect to variables. Since $F(0)$ denotes the case where $n = 1$, $F(0) = 0$ for both clauses and variables. Thus by (1), we have:

$\#C(TO_n) = 2^{r-1}(2^r + 2r - 1)$ $\quad$ O(n^2)
$\#V(TO_n) = r2^r$ $\quad$ O($n \log n$)

B. CNF size of Half Sorting Network

We denote by HM_n a HM having two inputs of length n. Here we count the number $F(r)$ of invoked HM_1s which are the minimum unit of HM. For $n = 2^r$, the number $g(r)$ of HM_1s the HM_n newly invokes is $2^r - 1 (= n - 1)$. Since $n = 1$ denotes HM_1 itself, $F(0) = 1$. So by (1), $F(r) = r2^r + 1$. Since a HM_1 consists of 3 clauses and 2 variables, we have:

$\#C(HM_n) = 3(r2^r + 1)$
$\#V(HM_n) = 2(r2^r + 1)$

Let HS_n be a procedure of HS for $n = 2^r$. Since HS_n uses a $HM_{n/2}$, the number $g(r)$ of HM_1s the HS_n newly invokes is $(r-1)2^{r-1} + 1$. Since sorting is unnecessary for $n = 1$, $F(0) = 0$. So by (1), $F(r) = 2^{r-1} r(r-1)/2 + 2^r - 1$. Thus we have:

$\#C(HS_n) = 3F(r)$ $\quad$ O($n \log^2 n$)
$\#V(HS_n) = 2F(r)$ $\quad$ O($n \log^2 n$)

C. CNF size of K-Cardinality Totalizer

We denote by SUA_k a SUA that outputs a sequence of at most k variables. The SUA_k generates the same clauses as the UA having the same inputs except that the clauses related to $r_i (i > k)$ are removed. Thus we have:

$\#C(SUA_k) = (m+1)(n+1) - 1 - i(i+1)/2$,
$\#V(SUA_k) = k$, where $i = m + n - k$.

Now, for $n = 2^r$ and $k = 2^s$, let us consider the case where the input n is divided into $n/k = 2^{r-s}$ blocks. Then a KTO_n invokes $(2^{r-s} - 1)$ SUA_ks and (2^{r-s}) TO_ks. Thus we have:

$\#C(KTO_n) = 2^r(2^s + s + 1) - 2^{s-1}(2^s + 3)$ $\quad$ O(nk)
$\#V(KTO_n) = (s+1)2^r - 2^s$ $\quad$ O($n \log k$)

D. CNF size of K-Cardinality Half Sorting Network

We denote by SM_n a SM having two inputs of length n. For $n = 2^r$, the number $g(r)$ of SM_1s the SM_n newly invokes to merge the results of two $SM_{n/2}$s is $2^{r-1} = (n/2)$. Since $n = 1$ denotes SM_1 itself, $F(0) = 1$. So by (1), $F(r) = (r+2)2^{r-1}$. Thus we have:

$\#C(SM_n) = 3(r+2)2^{r-1}$
$\#V(SM_n) = 2(r+2)2^{r-1}$

Authorized licensed use limited to: b-on: Universidade de Lisboa Reitoria. Downloaded on October 02,2025 at 15:05:38 UTC from IEEE Xplore. Restrictions apply.

Like KTO_n, for $n = 2^r$ and $k = 2^s$, a KHS_n invokes $(2^{r-s} - 1)$ SM_ks and (2^{r-s}) HS_ks. Thus we have:

$\#C(KHS_n) =$
$3(s^2 + s + 8)2^{r-2} - 3 \times 2^{r-s} - 3(s + 2)2^{s-1}$ $\quad O(n \log^2 k)$
$\#V(KHS_n) =$
$(s^2 + s + 8)2^{r-1} - 2 \times 2^{r-s} - (s + 2)2^s$ $\quad O(n \log^2 k)$

The above counting makes it clear that the Totalizer is superior to the Half Sorting Network in terms of the number of variables, while the latter is superior to the former in terms of the number of clauses. However, since a critical path for the Half Sorting Network is longer than that for the Totalizer, the delay of constraints propagation would become serious for the Half Sorting Network. In addition, we confirm that for both encoding schemes, the order of the number of generated CNF components can be reduced by making the K-Cardinality improvement.

The comparison of the numbers of CNF components generated by TO and HS is shown in Table I.

Table I
CNF SIZE (TO,HS).

		TO		HS	
$n = 2^r$	r	clauses	variables	clauses	variables
16	4	184	64	189	126
64	6	2,400	384	1,629	1086
128	7	9,024	896	4,413	2,942
256	8	34,688	2,048	11,517	7,678
512	9	135,424	4,608	29,181	19,454
1,024	10	534,016	10,240	72,189	48,126
2,048	11	2,118,656	22,528	175,101	116,734
4,096	12	8,435,712	49,152	417,789	278,526
8,192	13	33,656,832	106,496	983,037	655,358
16,384	14	134,438,912	229,376	2,285,565	1,523,710
65,536	16	2,148,499,456	1,048,576	11,993,085	7,995,390
262,114	18	34,364,325,888	4,718,592	60,948,477	40,632,318
1,048,576	20	549,776,261,120	20,971,520	301,989,885	201,326,590

IV. MODULO TOTALIZER

In this section, we propose a new CNF encoding scheme based on modulo computation, called a *Modulo Totalizer* (MTO). The Modulo Totalizer enables us to reduce the number of clauses, which is a drawback of the Totalizer. Here, we denote the integer n by the modulo number $q|_p r$ with modulus p such that $n = q \times p + r$. To maintain the arc consistency, we represent both the upper digits q and the lower digits p using a unary representation of integers.

A. Procedures for Modulo Totalizer

In modulo p computation, lower digits represent a remainder, while upper digits represent a quotient. Since the values the lower digits can take are $0 \dots p-1$, a carry to the upper digits occurs when the result of addition is greater than or equal to p. For example, $3+6 = 9$ is represented by modulo numbers with modulus 4 as $0|_4 3 + 1|_4 2 = 2|_4 1$.

The procedure for MTO is defined as shown in Figure 9. The MTO divides an input sequence $\langle i_1 \dots i_n \rangle$ into $\langle i_1 \dots i_{\lfloor n/2 \rfloor} \rangle$ and $\langle i_{\lfloor n/2 \rfloor + 1} \dots i_n \rangle$, and recursively calls two MTOs that take as input either of the subsequences. Then with a *Modulo Unary Adder* (MUA) that performs the addition of modulo numbers, the MTO generates a CNF formula Φ representing the addition of the results of two MTO calls.

$MTO(\langle i_1 \dots i_n \rangle, p)$ $(n \geq p)$
$= (\langle h_1 \dots h_{\lfloor n/p \rfloor} |_p r_1 \dots r_{p-1} \rangle, \Phi_1 \wedge \Phi_2 \wedge \Phi)$ **where**
$MTO(\langle i_1 \dots i_{\lfloor n/2 \rfloor} \rangle, p)$
$= (\langle f_1 \dots f_\alpha |_p a_1 \dots a_{p_1} \rangle, \Phi_1), \ (\alpha = \lfloor \lfloor n/2 \rfloor / p \rfloor)$
$MTO(\langle i_{\lfloor n/2 \rfloor + 1} \dots i_n \rangle, p)$
$= (\langle g_1 \dots g_\beta |_p b_1 \dots b_{p_2} \rangle, \Phi_2), \quad (\beta = \lfloor \lceil n/2 \rceil / p \rfloor)$
$MUA(\langle f_1 \dots f_\alpha |_p a_1 \dots a_{p_1} \rangle, \langle g_1 \dots g_\beta |_p b_1 \dots b_{p_2} \rangle)$
$= (\langle h_1 \dots h_{\lfloor n/p \rfloor} |_p r_1 \dots r_{p-1} \rangle, \Phi) \qquad p_1, p_2 < p$
$MTO(\langle i_1 \dots i_n \rangle, p)$
$= TO(\langle i_1 \dots i_n \rangle) = \langle h_0 |_p r_1 \dots r_n \rangle$ $(n < p) \quad h_0 = 0$

Figure 9. Modulo Totalizer.

Given inputs $\langle \ f_1 \dots f_\alpha \ |_p \ a_1 \dots a_{p_1} \ \rangle$ and $\langle \ g_1 \dots g_\beta \ |_p \ b_1 \dots b_{p_2} \ \rangle$, where $p_1, p_2 < p$, the MUA performs the unary addition on the lower digits and the upper digits respectively, then produces the CNF formulas Φ_1 and Φ_2 as shown in Figure 10. As for the addition of upper digits, the following two methods can be considered according to the treatment of a carry: Method A and Method B.

In Method A, we generate clauses representing the unary addition separately for each of the case where a carry occurs and the case where it does not. See Φ_2 of Figure 10.

$MUA(\langle f_1 \dots f_\alpha |_p a_1 \dots a_m \rangle, \langle g_1 \dots g_\beta |_p b_1 \dots b_n \rangle)$
$= \langle h_1 \dots h_\sigma |_p r_1 \dots r_{p-1} \rangle, \Phi_1 \wedge \Phi_2)$ **where**

$$\Phi_1 = \bigwedge_{i=0}^{m} \bigwedge_{j=0}^{n} \underset{i+j<p}{(\overline{a_i} \vee \overline{b_j} \vee r_{i+j} \vee c) \wedge} \underset{i+j>p}{(\overline{a_i} \vee \overline{b_j} \vee r_{mod(i+j,p)})} \wedge \underset{i+j=p}{(\overline{a_i} \vee \overline{b_j} \vee c)},$$

$$\Phi_2 = \bigwedge_{i=0}^{\alpha} \bigwedge_{j=0}^{\beta} (\overline{f_i} \vee \overline{g_j} \vee h_{i+j}) \wedge (\overline{c} \vee \overline{f_i} \vee \overline{g_j} \vee h_{i+j+1})$$

$a_0 = b_0 = r_0 = 1 \quad f_0 = g_0 = h_0 = h_{\sigma+1} = 1$

Figure 10. Modulo Unary Adder (A).

On the other hand, in Method B, we first perform the unary addition and output the result once in an intermediate variable h'_i, as shown in Figure 11. Then we distribute h'_i to an output variable h_i or a one digit higher one h_{i+1}, depending on having a carry or not. This method can reduce the number of generated clauses though it increases the number of generated variables, compared to Method A.

Authorized licensed use limited to: b-on: Universidade de Lisboa Reitoria. Downloaded on October 02,2025 at 15:05:38 UTC from IEEE Xplore. Restrictions apply.

$$\Phi_2 = \bigwedge_{i=0}^{\alpha}\bigwedge_{j=0}^{\beta} (\overline{f_i} \vee \overline{g_j} \vee h'_{i+j}) \bigwedge_{i=0}^{\alpha+\beta} (\overline{h'_i} \vee h_i) \wedge (\overline{c} \vee \overline{h'_i} \vee h_{i+1})$$
$$f_0 = g_0 = h_0 = h_{\sigma+1} = 1, \quad h'_0 = 1$$

Figure 11. Modulo Unary Adder (B).

B. CNF size of Modulo Totalizer

We evaluate the CNF size of cardinality constraints generated by MTO.

1) CNF size of MUA: In the lower digits computation, the MUA generates the same number of clauses as the UA that has two inputs of $p-1$ variables, and generates $p-1$ output variables as well as a carry variable. Thus the number of CNF components for the lower digits is as follows:

$\#C(MUA_L) = p^2 - 1$
$\#V(MUA_L) = p$

The computation of upper digits uses a procedure same as for UA. For inputs $\langle f_1 \dots f_\alpha \rangle$ and $\langle g_1 \dots g_\beta \rangle$, the number of clauses generated by UA is $\alpha\beta+\alpha+\beta$ and that of variables is $\alpha+\beta$. In Method A, since the clause generation is duplicated, the number of clauses is twice that for UA, and the number of variables is the same. In Method B, one requires $2(\alpha+\beta)$ clauses for distributing the intermediate variables. Thus we have:

$\#C(MUA_U) = 2(\alpha\beta + \alpha + \beta)$ (A) $\quad \alpha\beta + 3(\alpha + \beta)$ (B)
$\#V(MUA_U) = \alpha + \beta$ (A) $\quad 2(\alpha + \beta)$ (B)

2) CNF size of Upper,Lower,Base: The cardinality constraints generated by MTO consist of three parts: upper, lower, and basic. Here let the input n be $q \times p$ such that $q = 2^u, p = 2^v$. The lower part consists of $(q-1)$ MUA_ps. Thus we have:

$\#C(MTO_L) = (q-1)(p^2-1) = (2^u-1)(2^{2v}-1)$
$\#V(MTO_L) = (q-1)p = (2^u-1)2^v$

The procedure for the upper part is the same as that of TO. Let $F(u)$ denote the number of CNF components generated by the upper part when $q = 2^u$. For Method A, by $\#C(MUA_U)(A)$, we have $g(u) = 2((2^{u-1})^2 + 2^u)$ regarding clauses, while by $\#V(MUA_U)(A)$, we have $g(u) = 2^u$ regarding variables. When $u = 0$, the MTO generates nothing so that $F(0) = 0$ in either case. Thus by (1), we have:

$\#C(MTO_U) = 2^u(2^u + 2u - 1)$ (A)
$\#V(MTO_U) = u2^u$ (A)

Similarly, for Method B, we have $g(u) = (2^{u-1})^2 + 3(2^u)$ regarding clauses, while we have $g(u) = 2(2^u)$ regarding variables. Moreover, $F(0) = 0$ in either case. Thus for Method B, we have:

$\#C(MTO_U) = 2^{u-1}(s^u - 1) + 3u2^u$ (B)
$\#V(MTO_U) = 2u2^u$ (B)

Lastly, we consider the basic part based on a TO with a p-variables input. Since the basic part consists of (q) TO_ps, by $\#C(TO_p)$ and $\#V(TO_p)$, we have:

$\#C(MTO_B) = 2^u(2^{v-1}(2^v + 2v - 1))$
$\#V(MTO_B) = 2^u(v2^v)$

The CNF size of cardinality constraints for MTO is given by the sum of CNF components that the upper, the lower, and the basic parts generate as follows:

$\#C(MTO) =$
$\#C(MTO_L) + \#C(MTO_U) + \#C(MTO_B)$
$\#V(MTO) =$
$\#V(MTO_L) + \#V(MTO_U) + \#V(MTO_B)$

In this CNF encoding, the number of clauses generated by MTO is O($n^{3/2}$), while he number of variables generated by MTO is O($n \log n$).

When $n = 2^{20}$ and with Method B, $\#C(MTO) = 241,307,649$ and $\#V(MTO) = 7,798,720$. Comparing to TO, the MTO reduces the number of clauses to 1/2300, and the number of variables to 1/2.7. On the other hand, Comparing to HS, the MTO reduces the number of clauses to 1/1.3, and the number of variables to 1/26.

Table I summarizes the number of CNF components generated by MTO. Comparing to TO of Table I, when the number n of variable is about 1 million, $MTO(B)$ reduces the number of clauses to 1/2300, and the number of variables to 1/2.7. On the other hand, Comparing to HS, $MTO(B)$ reduces the number of clauses to 1/1.3, and the number of variables to 1/26.

Table II
CNF SIZE(MTO).

$n = 2^u * 2^v$	u	v	MTO(A)		MTO(B)	
			clauses	variables	clauses	variables
16	2	2	129	52	131	60
64	4	2	817	252	761	316
128	4	3	2,145	568	1,889	700
256	5	3	4,929	1,176	4,593	1,336
512	6	3	12,097	2,424	10,465	2,808
1,024	6	4	32,641	5,488	25,473	5,880
2,048	7	4	73,985	11,120	66,753	12,016
4,096	8	4	181,505	22,512	150,913	24,560
8,192	9	4	495,361	45,552	369,153	50,160
16,384	9	5	1,129,473	102,880	1,003,265	107,488
65,536	11	5	7,674,881	415,712	5,601,281	438,240
262,114	12	6	43,470,849	1,884,096	35,133,441	1,933,248
1,048,576	13	7	275,431,425	8,494,976	241,307,649	7,798,720

C. Correctness and efficiency

Now, we prove that the Modulo Totalizer is correct, and efficient in the sense that unit propagation can restore the generalized arc consistency. The correctness and efficiency of the Totalizer was already proven in [3]. For comparison, we adopt the notations defined in [3].

We denote the truth values $TRUE$ and $FALSE$ by 1 and 0, respectively. An *instantiation* or a *truth assignment* I of a set of propositional variables is a function that maps

Authorized licensed use limited to: b-on: Universidade de Lisboa Reitoria. Downloaded on October 02,2025 at 15:05:38 UTC from IEEE Xplore. Restrictions apply.

each variable $v \in V$ to a non empty set $I(v) \subseteq \{0,1\}$. A variable v is said to be *fixed* to 0 or *assigned the value* 0 by an instantiation I if $I(v) = \{0\}$, fixed to 1 if $I(v) = \{1\}$, and free $I(v) = \{0,1\}$. We simply denote $I(v) = \{1\}$ by $v = 1$, $I(v) = \{0\}$ by $v = 0$. An instantiation I of V is said to be *complete* if it fixes all the variables in V; otherwise it is said to be *partial*. The above notations are extended to a pair (U, V) of sets of variables.

A *unit clause* is a clause in which all literals are fixed to 0 but one which is undefined. *Unit propagation* denotes the process that fixes the undefined variable in a unit clause so as to satisfy it, until the empty clause is produced or no unit clause remains.

A Modulo Totalizer counts the number of 1's in the set $E = \{e_1, \ldots, e_n\}$ of input variables, and outputs the results to the sets $U = \{u_1, \ldots, u_q\}$ and $L = \{l_1, \ldots, l_{p-1}\}$, representing modulo unary integers, where $q = \lfloor n/p \rfloor$.

A partial instantiation of (U, L) is said to be *pre-unary* if for each $u_i = 1$, $u_j = 1$ $(j < i)$ and for each $l_i = 1$, $l_j = 1$ $(j < i)$. Any partial pre-unary instantiation I of (U, L) is related to an integer range. The bounds of this range will be denoted $min(I)$. When $n = 7, p = 4$, if I is a partial instantiation such that $u_1 = 1, l_1 = l_2 = 1$, and u_2, l_3 are free, then $min(I_U) = 1$ and $min(I_L) = 2$.

Lemma 4.1: (**forward propagation**) Let $E = \{e_1, \ldots, e_n\}$ be a set of input variables to represent the integer n in unary, and $(U = \{h_1, \ldots, h_q\}, L = \{r_1, \ldots, r_{p-1}\})$ be a pair of the sets of output variables to represent the integer $n = q \times p + r$ in modulo unary. Let $MTO(E, p)$ be a Modulo Totalizer with modulus p that takes an input E and produces an output (U, L). If $m = u \times p + l$ input variables are fixed to 1, and all the other variables of the Modulo Totalizer are free, then the partial instantiations I_U, I_L of U, L obtained after unit propagation in $MTO(E, p)$ are pre-unary and such that $min(I_U) = u$ and $min(I_L) = l$.

Proof: By induction on the number of input variables.

For $n < p$, since $MTO(E, p) = TO(E)$, the property is obvious by the forward propagation property of a Totalizer. Thus, we take the $TO(E)$ as a base case.

Now let us consider the property holds for any $k < n$.

Let $(U_A = \{f_1, \ldots, f_{q_A}\}, L_A = \{a_1, \ldots, a_{v_A}\})$ be the modular unary representation of $\lfloor n/2 \rfloor$ associated to the first child of the root, and $(U_B = \{g_1, \ldots, g_{q_B}\}, L_B = \{b_1, \ldots, b_{v_B}\})$ be the modular unary representation of $n - \lfloor n/2 \rfloor$ associated to the other child where $v_A, v_B \leq p - 1$.

Considering the leaves of the subtree below each child of the root, the input set E is divided into two disjoint subsets E_A and E_B. Let $m_A = u_A \times p + l_A$ denote the number of variables fixed to 1 in E_A and $m_B = u_B \times p + l_B$ denote the number of variables fixed to 1 in E_B. Clearly, we have $m = m_A + m_B$. It follows from the induction hypothesis that I_{U_A}, I_{L_A}, I_{U_B} and I_{L_B} are pre-unary, $min(I_{U_A}) = u_A$, $min(I_{L_A}) = l_A$, $min(I_{U_B}) = u_B$, $min(I_{L_B}) = l_B$.

[**Lower digits**]

For $l_A + l_B < p$, we have $l = l_A + l_B$. It is easy to see that for every t such that $1 \leq t \leq l$ there exist t_A, $t_A \leq min(I_{L_A})$, and t_B, $t_B \leq min(I_{L_B})$ such that $t = t_A + t_B$. Thanks to the clause $\overline{a_{t_A}} \vee \overline{b_{t_B}} \vee r_{t_A+t_B} \vee c$, unit propagation fixes to 1 the variable r_t where a carry variable c is to be fixed to 0 by variable decision in False preference mode.

For $l_A + l_B = p$, we have $l = 0$. In this case, c is fixed to 1 by the clause $\overline{a_{l_A}} \vee \overline{b_{l_B}} \vee c$.

For $l_A + l_B > p$, we have $l = \mathrm{mod}(l_A + l_B, p)$. For every t such that $1 \leq t \leq l$ there exist t_A, $t_A \leq min(I_{L_A})$, and t_B, $t_B \leq min(I_{L_B})$ such that $t = \mathrm{mod}(t_A + t_B, p), t_A + t_B > p$. Thanks to the clause $\overline{a_{t_A}} \vee \overline{b_{t_B}} \vee r_{\mathrm{mod}(t_A+t_B,p)}$, unit propagation fixes to 1 the variable r_t.

[**Upper digits**]

Since the computation of upper digits is performed by essentially the same scheme as Totalizer, the forward propagation property holds. ∎

Lemma 4.2: (**backward propagation**) Let $MTO(E, p)$ be a Modulo Totalizer with modulus p that takes an input E and produces an output (U, L). If $m = u \times p + l$ input variables are fixed to 1, and the remaining input variables being free, and the output variables $r_{l+1}, \ldots, r_{p-1}$ and $h_{u+1} \ldots, h_q$ are fixed to 0, then all the input variables remaining free are instantiated to 0 by unit propagation.

Proof: By induction on the number of input variables.

For $n < p$, since $MTO(E, p) = TO(E)$, the property is obvious. So, we assume that $n \geq p$.

If we consider solely the assignment to the input variables, the unit propagation assigns to the output variables the values such that $min(I_U) = u$ and $min(I_L) = l$.

[**Lower digits**]

Suppose that variables r_{l+1} to r_{p-1} are assigned the value 0. Let t be any integer such that $1 \leq t \leq p - 1 - l$.

For $l_A + l_B < p$, $l = l_A + l_B$. Clearly we have $r_{l+t} = 0$.

Let i, j be any integers such that $1 \leq i \leq v_A - l_A$, $0 \leq j \leq l_B$, $l_A + i + j = l + t$. If $l_A + i \leq v_A$, there is a clause $\overline{a_{l_A+i}} \vee \overline{b_j} \vee r_{l_A+i+j}$. By unit propagation, this clause assigns the value 0 to a_{l_A+i}. By the same argument we can prove that every b_{l_B+i} such that $l_B + i \leq v_B$ is also assigned the value 0.

For $l_A + l_B \geq p$, we have $l = \mathrm{mod}(l_A + l_B, p)$. Note that $l < l_A, l_B$. Clearly we have $r_{l+t} = r_{\mathrm{mod}(l_A+l_B+t,p)} = 0$. If $l_A + t \leq v_A$, by unit propagation, the clause $\overline{a_{l_A+t}} \vee \overline{b_{l_B}} \vee r_{\mathrm{mod}(l_A+l_B+t,p)}$ assigns the value 0 to a_{l_A+t}. By the same argument, it is shown that every variable b_{l_B+t} such that $l_B + t \leq v_B$ is also assigned the value 0. By the induction hypothesis, all the free variables below the nodes A and B are assigned 0.

[**Upper digits**]

Every MUA associated to a leaf performs the addition on the fixed values of carry variables. Since the computation of upper digits is essentially performed by a Totalizer, if the variables $h_{u+1} \ldots, h_q$ are assigned the value 0, all the free

Authorized licensed use limited to: b-on: Universidade de Lisboa Reitoria. Downloaded on October 02,2025 at 15:05:38 UTC from IEEE Xplore. Restrictions apply.

variables f_i or g_i appearing in the path to a leaf are assigned 0 due to the clause $\overline{f_i} \vee \overline{g_j} \vee h_{i+j}$. The same may be said for the case where $\overline{c} \vee \overline{f_i} \vee \overline{g_j} \vee h_{i+j+1}$ is used. ∎

The comparator

The comparator is a set of unit and binary clauses that are satisfied if and only if the conjunction of the input variables of the Modulo Totalizer that matches with the cardinality constraint. Then the constraint $N(E_I) \leq \rho = \sigma \times p + \nu$ will be specified as follows:

$$\bigwedge_{\sigma+1 \leq i \leq q} (\overline{h_i}) \qquad \bigwedge_{\nu+1 \leq j \leq p-1} (\overline{h_\sigma} \vee \overline{r_j})$$

Theorem 4.1: The CNF encoding of a cardinality constraint with a Modulo Totalizer is correct and efficient.

Proof:

Correctness: Lemma4.1 ensures that the instantiation of the output variables is the modular unary value of the number of input variables fixed to 1. Then the conjunction of the Modulo Totalizer and the comparator is satisfiable if and only if the number of input variables fixed to 1 is less than or equal to $\rho = \sigma \times p + \nu$.

Efficiency: Let $m = u \times p + l$ be the number of input variables fixed to 1, and ρ be the upper bound of the range encoded by the comparator. It follows from Lemma 4.1 that unit propagation fixes h_1 to h_u and r_1 to r_l to 1. In addition, the clauses of the comparator allow unit propagation to fix $h_{\sigma+1}$ to h_q to 0 and to fix $r_{\nu+1}$ to r_{p-1} to 0.

If $m > \rho$ then unit propagation fixes h_σ and $r_{\nu+1}$ to 1, which conflict with the clause $(\overline{h_\sigma} \vee \overline{r_j})$ of the comparator, thus the empty clause is produced.

If $m = \rho$ then unit propagation fixes h_σ and r_ν to 1. Moreover, because output variables $h_{\sigma+1}$ to h_q and $r_{\nu+1}$ to r_{p-1} are fixed to 0 by the clauses of the comparator, it follows from Lemma 4.2 that unit propagation fixes the free input variables to 0. ∎

D. Remarks

- TO counts the number of 1's and 0's using two UA clauses representing $\sigma \geq \alpha + \beta$ and $\sigma \leq \alpha + \beta$. HS also does the same using 6 corresponding HM clauses. For counting both 1's and 0's, TO and HS duplicate clauses but need not duplicate variables.
- MTO counts the number of 1's using 5 MUA clauses representing $\sigma \geq \alpha + \beta$. In modulo unary representation, we can express only the number of 1's. To count 0's, we give the negation of the input variables to MTO. Thus, when counting both 1's and 0's, we need to duplicate variables as well as clauses.
- We could obtain a *Moodulo based Half Sort* (MHS) by implementing the MUA using HM. However, the numbers of clauses and variables for MHS are still $O(n\log^2 n)$.

V. Experimental Evaluation

In this section, we evaluate the performance of a MaxSAT solver MTOM that employs MTO. We have compared it with other MaxSAT solvers using different encodings. Specification of the computer used in this experiment is an Intel Core i5 CPU 750 2.67GHz, 7.8GB RAM and Linux. The set of instances (a total of 876 instances) we used comes from benchmarks of crafted and industrial subcategories of Partial Max-SAT category in Max-SAT 2012.We define below the abbreviations of the MaxSAT solvers used in the evaluation:

- QM: QMaxSAT
- KQM: K-Cardinality QMaxSAT
- MTOM(A): MTO(A) based MaxSAT solver
- MTOM(B): MTO(B) based MaxSAT solver

Each MaxSAT solver uses MiniSat 2.2[9] as inference engine. QM is a MaxSAT solver that employs a TO to encode cardinality constraints. KQM is a K-Cardinality version of QMaxSAT.

A. Comparison of CNF size

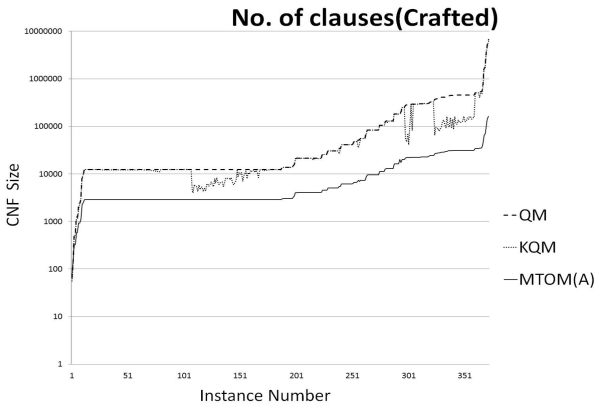


Figure 12. CNF Size (No. of Cls) for Crafted

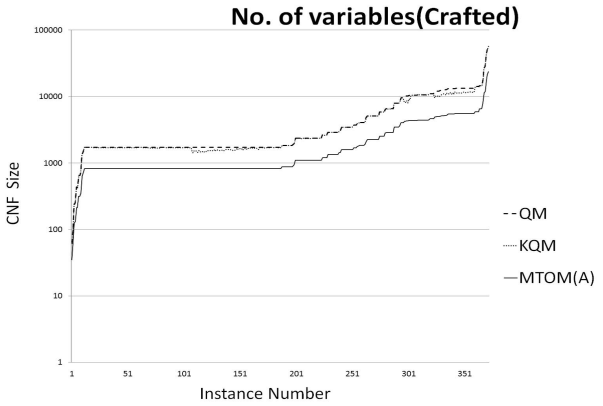


Figure 13. CNF Size (No. of Vars) for Crafted

First, we compare the numbers of variables and clauses generated by

Authorized licensed use limited to: b-on: Universidade de Lisboa Reitoria. Downloaded on October 02,2025 at 15:05:38 UTC from IEEE Xplore. Restrictions apply.

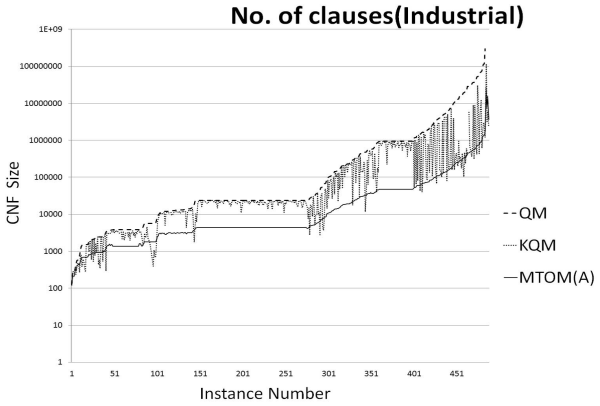


Figure 14. CNF Size (No. of Cls) for Industrial

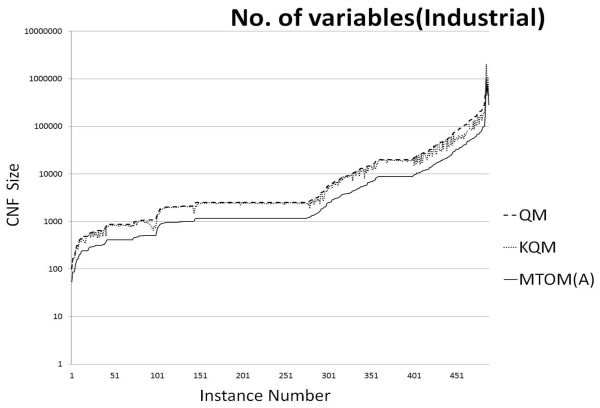


Figure 15. CNF Size (No. of Vars) for Industrial

MaxSAT solvers QM, KQM, and MTOM. The experimental results are shown in Figures 12-15, where MTOM(B) is omitted because the difference with MTOM(A) is not so large. In each figure, the horizontal axis represents the instance number and the vertical axis represents the size of generated CNF using a common logarithm.

Figures 12 and 13 show that in crafted instances, MTOM(A) generates fewer clauses and variables than others to encode cardinaliy constraints. As shown in Figures 14 and 15, in industrial instances, MTOM(A) always generates fewer variables than others whereas there are several instances in which KQM generates fewer clauses due to K-Cardinality effects. However, MTOM(A) generates fewer clauses for most instances,

and even when it generates more clauses than KQM, the difference of that case is small. Thus, we can say that MTO overcomes the disadvantage of TO.

B. Comparison of runtime

Figures 16 and 17 show the runtime of each MaxSAT solver. For each run, the time limit was set to 1200 seconds. Here, the horizontal axis represents the number of solved instances and the vertical axis represents the elapsed time.

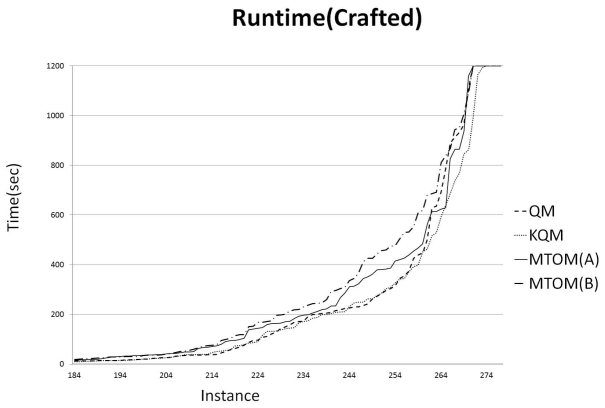


Figure 16. Execution Time for Crafted

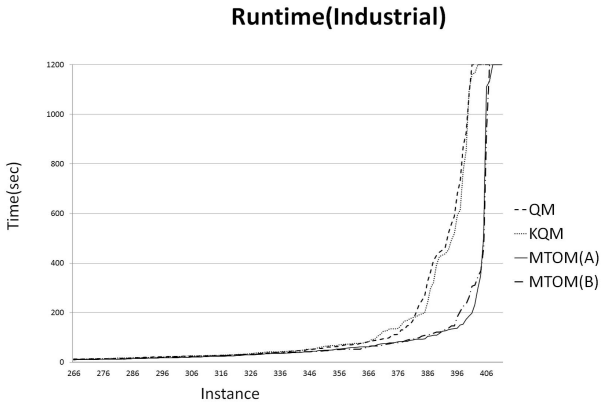


Figure 17. Execution Time for Industrial

In crafted instances, although MTOM(A) and MTOM(B) solved fewer problems than others at 400 seconds, the difference between them decreases as time passes, and both MTOMs finally approached to three instances behind KQM. In industrial instances, Both MTOM(A) and MTOM(B) always solved more instances than the others at any time. MTOM(A) performed best in terms of the total number of solved instances, which solved 677 instances of the two subcategories.

VI. Conclusion

In this paper, as CNF encoding schemes of cardinality constraints, we have introduced Totalizer, Half Sorting Network, and K-cardinality versions for both, as well as the

Authorized licensed use limited to: b-on: Universidade de Lisboa Reitoria. Downloaded on October 02,2025 at 15:05:38 UTC from IEEE Xplore. Restrictions apply.

compact procedures for them. Then, we have given the method to count the exact number of CNF components generated by each encoding scheme uniformly.

To make use of an advantage and overcome the drawback of Totalizer, we have proposed a Modulo Totalizer based on modulo computation, and proved the arc consistency of it. By exactly counting, we have shown that with modulus $p = n^{1/2}$ for input n, the number of clauses generated by the Modulo Totalizer is O($n^{3/2}$), while that of variables is O($n \log n$). To evaluate our scheme, we tested it on MaxSAT benchmarks. Experimental results show that the CNF size of cardinality constraints decreases dramatically by the Modulo Totalizer, and that the Modulo Totalizer achieves performance comparable to or higher than K-Cardinality Totalizer in terms of runtime.

For future work, we are going to develop a 2-level Modulo Totalizer that deals with a modulo number $h \mid_{pq} u \mid_p r = h \times pq + u \times p + r$ for reducing the CNF size further more, and to perform K-Cardinality improvements on the Modulo Totalizer for further performance enhancement.

ACKNOWLEDGMENT

This work was supported by JSPS KAKENHI Grant Numbers 25330085, 25330262.

REFERENCES

[1] A. Biere, M. Heule, H. Maaren, and T. Walsh. *Handbook of Satisfiability*. IOS Press, 2009.

[2] J. Marques-Silva, I. Lynce, and S. Malik. Conflict-Driven Clause Learning SAT Solver. chapter 4, pages 131-153. Volume 185 of Biere et al. [1], 2009.

[3] O. Bailleux, and Y. Boufkhad. Efficient CNF Encoding of Boolean Cardinality Constraints. In *Proc. of 9th International Conference on Principles and Practice of Constraint Programming (CP'03)*, pages 108-122, 2003.

[4] R. Asín, R. Nieuwenhuis, A. Oliveras, and E. Rodríguez-Carbonell. Cardinality networks: a theoretical and empirical study. Constraints 16. 2 pages 195-221, 2011.

[5] Z. Fu, and S. Malik. On solving the partial MAX-SAT problem. In *Proc. of 9th International Conference on the Theory and Applications of Satisfiability Testing (SAT'06)*, pages 252-265, 2006.

[6] M. Koshimura, T. Zhang, H. Fujita, and R. Hasegawa. QMaxSAT: A Partial Max-SAT Solver. *Journal on Satisfiability, Boolean Modeling and Computation*, 8, pages 95-100, 2012.

[7] D. Le Berre, and A. Parrain. The Sat4j library, release 2.2 system description. *Journal on Satisfiability, Boolean Modeling and Computation*, 7, pages 59-64, 2010.

[8] J. Argelich, C. Li, F. Manyà, and J. Planes. The first and second Max-SAT evaluations. *Journal on Satisfiability, Boolean Modeling and Computation*, 4, pages 251-278, 2008.

[9] N. Eén, and N. Sörensson. An extensible SAT-solver. In *Proc. of 6th International Conference on the Theory and Applications of Satisfiability Testing (SAT'03)*, pages 502-518, 2004.

[10] C. Sinz. Towards an optimal CNF encoding of boolean cardinality constraints. In *Proc. of 11th International Conference on Principles and Practice of Constraint Programming (CP'05)*, pages 827-831, 2005.

[11] J Marques-Silva, and I Lynce. Towards robust CNF encodings of cardinality constraints. In *Proc. of 13th International Conference on Principles and Practice of Constraint Programming (CP'07)*, pages 483-497, 2007.

[12] M Codish, and M Zazon-Ivry. Pairwise Cardinality Networks. In *Proc. of 16th International Conferences on Logic for Programming, Artificial Intelligence and Reasoning (LPAR-16)*, pages 154-172, 2010.

Authorized licensed use limited to: b-on: Universidade de Lisboa Reitoria. Downloaded on October 02,2025 at 15:05:38 UTC from IEEE Xplore. Restrictions apply.